

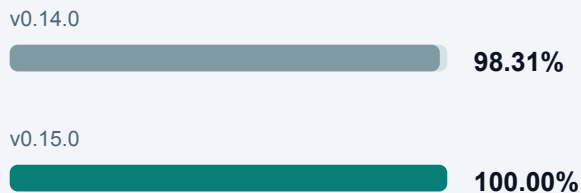
INVARUNE BY NIMESHBUILD

Broader inspection. Measured against the same evidence.

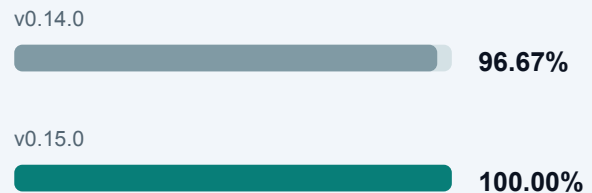
Benchmark update / Invarune 0.15.0

The unchanged development-visible fixture corpus and eight pinned public projects were scanned again after targeted detector fixes. This publication records the measured changes, comparison scope and unresolved work.

Precision



Recall



Unchanged fixture comparison	Observed result
Cases / original corpus	113 / 1.2.0
Previously failing cases now correct	3
New mismatches on previously correct cases	0
Remaining labeled mismatches	0
Pinned public-project scope	8 projects / 4,120 exported files
Current Invarune source observations / coverage gaps	147 / 24

These percentages describe project-authored rule-presence labels used during development. They are not production accuracy, independent validation or a claim that Invarune is better than every scanner.

[Interactive benchmark dashboard and downloadable evidence](#)

Harder unseen cases still expose limits

First blind 32-case result: 4 TP / 16 TN / 0 FP / 12 FN. After disclosure and fixes: 16 TP / 16 TN / 0 FP / 0 FN. A separate eight-case holdout still has 2 false negatives. The original 331 skill/tool labels retain one false-positive score. These separate denominators are detailed inside.

Read the evidence in layers

Fixture accuracy and unchanged labels

What was fixed and what still needs work

Source scope and release additions

A harder challenge exposed a real gap

Label changes and a second sealed check

Fresh comparison on eight public projects

Finding overlap and complementary coverage

Why use invscan in an agent/MCP review

Reproduce, validate and inspect the complete ledger

Evidence layer	What it establishes
Deterministic fixtures	Agreement with explicitly authored rule-presence labels; supports regression and before/after analysis.
Public source observations	Patterns reported by configured tools on exact pinned files; requires source and deployment review.
Optional model review	Advisory judgments only when actually requested and completed; not a replacement for independent truth.
Runtime validation	Still needed for authentication, reachability, tenant isolation, prompt-injection resistance and deployed safeguards.

All measurements in this update preserve target isolation: no upstream project code, service or exploit was executed. Tool support, parser warnings and incomplete work remain visible.

Fixture accuracy

The before and after runs use the exact same 113 cases, source bytes, paths and expected labels. Corpus SHA-256:

eb7f1eba93f8e9842634cbd12687dde5bea879505d99d367621214583e54dde3

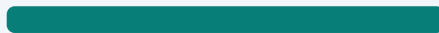
Precision

v0.14.0



98.31%

v0.15.0



100.00%

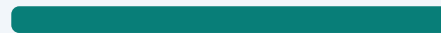
Recall

v0.14.0



96.67%

v0.15.0



100.00%

Outcome	Before 0.14.0	After 0.15.0
True positive	58	60
True negative	52	53
False positive	1	0
False negative	2	0
Analysis errors	0	0

Precision = $TP / (TP + FP)$. Recall = $TP / (TP + FN)$. A true-positive label here means that the designated detector correctly recognized the fixture's specified source pattern. It is not an independently confirmed vulnerability.

All challenge failures remain in these denominators. Default regression gating is a narrower CI policy and must not be read as zero mistakes. Fixes were developed with knowledge of these cases; this is a transparent development benchmark, not a held-out test.

[Before accuracy JSON](#)

[After accuracy JSON](#)

Fixes and remaining limits

Previously failing cases now agree with the unchanged label

Fixture	Bounded detector improvement
gap-interprocedural-url	Propagate known input through bounded local Python arguments and returns.
gap-js-wrapper-taint	Summarize supported compact JavaScript wrappers and connect supplied arguments to sinks.
gap-validated-user-url	Recognize a supported reject branch that restricts the remaining URL to a fixed destination.

No remaining mismatch on these 113 assertions

The separate blind challenge and development extension corpora still contain misses. Matching this older, development-visible corpus does not establish complete security coverage.

The release's paired and adversarial regression tests cover the supported interpretations and counterexamples. General interprocedural flow, dynamic runtime values, full YAML semantics and arbitrary shell expansion are not proven by these targeted changes. Review an uncertain finding in context instead of silently suppressing it.

[Detailed change analysis and remaining gaps](#)

Skills, tools and honest scope

On the unchanged 4,120 exported files, Invarune observations changed from 146 to 147. The final scan retains 24 analysis gaps. New metadata inspection can expose additional unknowns without changing the finding count.

Instruction checks and expanded deterministic scope

Rule	Risk evidence
AI043	Explicit instruction-hierarchy overrides in supported skill/tool text.
AI044	Sensitive-data transfer instructions with an explicit destination.
AI045	Concealed actions and approval/security-boundary bypass directives.
AI046	Read-only hints conflicting with a destructive description or supported direct handler write.
AI047	Explicit other-write permission bits on recognized filesystem calls.

Separate measurement: 81 cases / 331 rule-presence assertions. TP 43; TN 287; FP 1; FN 0. Precision 97.73%; recall 100.00%. All authored challenge misses remain in these denominators.

The separate skill/tool corpus is not added to the original 113 assertions. Each label describes a source pattern, not malicious authorship or exploitation. The public-source export excludes most skill documentation and cannot measure complete skill-package coverage.

[Separate skill/tool measurement and known misses](#)

[Every scan: algorithms, controls, sources and uncertainty](#)

Interpreting a score

There is no overall security percentage. Findings use exact severity counts. Partial mapping reach = active selected controls with an active mapped rule / active selected controls. Optional AI answer coverage = uniquely answered active checks / active selected checks. Both are multiplied by 100; zero denominators are null. Neither is a control pass rate.

Exceptions receive no pass credit. AI may add advisory concerns or help investigate inconclusive evidence; it cannot clear static findings or coverage gaps. Missing runtime proof remains unresolved.

[Exact source observation delta](#)

A harder test exposed a real gap

A separate benchmark assistant sealed 32 cases before the first candidate freeze: 16 source-flow cases and 16 literal tool descriptions. The first result remains visible. It revealed that declared tool-handler parameters were not being treated as endpoint input, and that several composed instructions were missed.

Same 32-case challenge	TP	TN	FP	FN
Earlier v0.14 release	3	16	0	13
First blind v0.15 candidate	4	16	0	12
After disclosure and fixes	16	16	0	0

What changed after disclosure

The implementation work targets recognized MCP endpoints, bounded local argument flow, operative instruction composition and defensive counterexamples. The third row is development performance because the cases and labels were disclosed before those fixes. It must never be presented as a second blind result.

Equivalent offline metadata input

Same 16 descriptions	TP	TN	FP	FN
Invarune final deterministic	8	8	0	0
Cisco MCP Scanner: YARA only	0	8	0	8

Cisco's enabled YARA engine also passed a separate known-positive/safe sanity pair. Its API, LLM and behavioral analyzers were not enabled. These rows do not compare Cisco's full product to Invarune or establish production prompt-injection resistance.

Separate four sealed descriptions	TP	TN	FP	FN
Invarune first execution	0	2	0	2
Cisco MCP Scanner: YARA only	1	1	1	1

Sealed inputs, first result and all post-disclosure outcomes

Peer scope, official sources and benchmark design

Keep labels and uncertainty visible

A second, smaller eight-case set was sealed after the first challenge was disclosed and before the detector freeze. It uses four source cases and four metadata cases. This is a within-project temporal holdout, not third-party independent validation or a statistically representative sample.

Final sealed confirmation	TP	TN	FP	FN
Eight cases, first execution	2	4	0	2

Two misses remain open: a .env contents-transfer instruction and a precedence instruction referring to a system message. A report-presentation-only release change required a repeat; all eight detector outcomes stayed identical. That repeat is not a new blind test.

One old scope label was explicitly corrected

The legacy skill corpus labels an override inside a tool input-schema field as out of scope. The expanded detector inspects model-visible schema descriptions. The old 331 assertions remain untouched and scored; a separate revision changes only that label, preserving all 81 source inputs.

331-assertion label set	TP	TN	FP	FN
Original legacy labels	43	287	1	0
Explicitly corrected label revision	44	287	0	0

Source observations are a different question

Twelve purposively selected source locations were inspected by the benchmark assistant. The optional AutoGen execution directory is made world writable; AI047 now checks the other-write permission bit. Other peer-only observations concern safe YAML loaders, serialization, public OAuth URLs or existing archive checks. None of this establishes an aggregate deployed-vulnerability true-positive rate.

[Twelve pinned source reviews and qualifications](#)

[Explicit corpus-label correction and both paired outcomes](#)

Runtime authorization, model behavior, DNS/redirect effects and deployed isolation remain separate evidence. Optional AI investigation is advisory and cannot erase deterministic findings.

Eight projects. Fresh executions.

The same exported first-party source/configuration bytes were offered to Invarune, Semgrep CE, Bandit and Gitleaks. Equal input does not make their languages, rule scope or supported inputs identical. Counts below are observations, not unique vulnerabilities.

Project	Files	Invarune	Semgrep	Bandit	Gitleaks
autogen	786	12	6	183	0
crewai	968	27	8	232	0
fastmcp	283	0	2	95	1
github-mcp	163	15	1	unsupported	0
langgraph	211	23	8	84	1
mcp-reference	86	61	0	0	0
openhands	1237	9	0	unsupported	1
pydantic-ai	386	0	2	344	0

Tool	Recorded version	Observations	Scope status
Invarune	0.15.0	147	1 completed; 7 completed_with_analysis_gaps
Semgrep CE	1.177.0	27	1 completed_no_findings; 3 completed_with_analysis_errors; 4 completed_with_findings
Bandit	1.9.4	938	1 completed_no_findings; 1 completed_with_analysis_errors; 4 completed_with_findings; 2 unsupported_no_python_files
Gitleaks	8.30.1	3	5 completed_no_findings; 3 completed_with_findings

Invarune reports 24 coverage gaps. Those inputs are incomplete, not clean. Semgrep parser warnings, Bandit parser errors and unsupported exports are retained in the run-status receipts. All eight Invarune projects have two byte-identical repeated outputs under the recorded runtime and settings.

[All normalized run statuses and coverage details](#)

What overlaps - and what does not

A compatible family, the same pinned project/path and intersecting line spans are required for a match. One-to-one pairs require exactly one candidate on each side. Ambiguous and location-only relations remain separate; matching does not establish a true-positive label.

Compared with	1:1 pairs	Ambiguous edges	Invarune unmatched	Other tool unmatched
Semgrep CE	7	0	140	20
Bandit	13	5	129	924
Gitleaks	1	0	146	2

The complete ledger exposes what each scanner reported, the other-tool match candidates, exact pinned paths and rule-family predicates. A tool-only record may reflect useful additional scope, a differently defined rule, unsupported analysis, a false alarm or a genuine missed risk; count alone does not decide which.

Native findings remain review signals

Broad source scanners can surface assertions, weak-hash uses, serialization APIs and import-only records. Invarune emphasizes selected agent/MCP configuration and execution risks with control/source mappings. Secret scanners, dependency/CVE scanners and runtime/adversarial tests add other evidence. No tool is awarded false positives merely because another tool lacks the same rule.

No new ground-truth percentage is claimed for public projects

This deterministic rerun does not perform Claude adjudication or independent exploit validation. Earlier model-authentication failures remain historical failures, and earlier selected source audits remain separate evidence. Confirmed-vulnerability precision and public-project recall are unavailable.

[Complete finding-by-finding ledger](#)

[Machine-readable overlap and unmatched sets](#)

An agent/MCP review workflow

Invarune's value is the traceable review workflow around selected security checks. These are product capabilities documented and tested by this project, not a score against untested competitor editions.

Capability	How to use the evidence
Source or built Linux image	Inspect supported source/configuration and image metadata without starting target code or a container. Compiled-only logic remains unassessed.
Offline control explorer	Use <code>invscan --ask</code> and explain commands without a model, login or network. Each answer explains control meaning, sources and limits.
47 rules / 66 controls / 132 checks	Rules partially map to 30 controls; the remaining controls and complete acceptance checks need other evidence. No inferred compliance pass.
Concrete remediation	Reports connect findings to agent/MCP context, proposed fixes, verification steps and possible mitigating layers. Undeployed defenses do not lower risk automatically.
Optional bounded AI review	Enable official CLI or API/gateway review explicitly. Deterministic validation constrains evidence/citations/budgets; model judgments stay advisory.
Review and justification roundtrip	Carry edited supported report inputs into a fresh scan. Accepted justifications stay justified, excluded from the applicable scored denominator, and retain evidence bindings.

Use complementary runtime validation for MCP authorization, tenant separation, sandbox escape resistance, egress enforcement and prompt-injection task outcomes. A repository or image cannot establish the deployed identity, network policy or human approval process.

[CLI quick start](#)

[Controls, benchmarks and primary source mappings](#)

Reproduce and audit the result

Every published count is derived from the linked local execution artifacts. The PDF builder rejects mismatched fixture hashes/labels, invalid ledger totals, overlap partition errors, mismatched scanner versions and changed before/after project inputs.

Input	SHA-256
accuracy-before.json	4d8c0f5213eedb3879986665757e7f5e17689dd26417642df7fa8b2d62874a27
accuracy-after.json	1434443d45b23804f1da6b2f9ed5e33f6732b0d7587bde9ee8eb31045e6363f8
observations.json	e7c85c4a934c2d5b70873c74ea594f919b537093debcdb44e3a6bd40c6e58579
overlaps.json	f5d2c60a2b254410878a6b9387a68a5076c7a001f93f357a4be0982aa343bd96
run-status.json	06d487b0e6965e33af2ae3fb53e1de479dea23085769e40e1305e21b3be140f6
baseline-identity.json	76c33feaf55ed689e3c8581484737b7193caac710c5ef4beaaf23753c72f50db
before-after-accuracy.json	6e167febd3fc9e79d8aa00a2f82516f5182057b4944681e7325514050f5afe37

Reproduction sequence

1. Read this benchmark README and exact tool/source lock.
2. Recreate the pinned source exports and verify their manifests.
3. Run the frozen baseline and final scanner independently; preserve both receipts.
4. Run the recorded external versions and rule pack on the same exports.
5. Regenerate the exhaustive observation/overlap ledger and fixture evaluations.
6. Build the dashboard/PDF from those receipts, then render and visually inspect every page.

[Reproduction protocol and tool configuration](#)

[Source selection and pinned repositories](#)

[PDF and site authoring instructions](#)

Invarune by NimeshBuild is an original engineering synthesis. Referenced publishers and compared projects do not endorse it. Third-party materials retain their own terms. Public repository visibility does not independently grant an open-source license.